

Lecture 19

Compiler Design CSC-305

Awanish Pandey

Department of Computer Science and Engineering
Indian Institute of Technology
Roorkee

September 8, 2026

Bottom-up evaluation of S-attributed definitions

- Can be evaluated while parsing

Bottom-up evaluation of S-attributed definitions

- Can be evaluated while parsing
- Whenever reduction is made, value of new synthesized attribute is computed from the attributes on the stack

Bottom-up evaluation of S-attributed definitions

- Can be evaluated while parsing
- Whenever reduction is made, value of new synthesized attribute is computed from the attributes on the stack
- Extend stack to hold the values also

Bottom-up evaluation of S-attributed definitions

- Can be evaluated while parsing
- Whenever reduction is made, value of new synthesized attribute is computed from the attributes on the stack
- Extend stack to hold the values also
- The current top of stack is indicated by `ptr top`

Bottom-up evaluation of S-attributed definitions

- Can be evaluated while parsing
- Whenever reduction is made, value of new synthesized attribute is computed from the attributes on the stack
- Extend stack to hold the values also
- The current top of stack is indicated by `ptr` top
- Suppose semantic rule $A.a = f(X.x, Y.y, Z.z)$ is associated with production $A \rightarrow XYZ$

Bottom-up evaluation of S-attributed definitions

- Can be evaluated while parsing
- Whenever reduction is made, value of new synthesized attribute is computed from the attributes on the stack
- Extend stack to hold the values also
- The current top of stack is indicated by `ptr` top
- Suppose semantic rule $A.a = f(X.x, Y.y, Z.z)$ is associated with production $A \rightarrow XYZ$
- Before reducing XYZ to A, value of Z is in $val(top)$, value of Y is in $val(top - 1)$ and value of X is in $val(top - 2)$

Bottom-up evaluation of S-attributed definitions

- Can be evaluated while parsing
- Whenever reduction is made, value of new synthesized attribute is computed from the attributes on the stack
- Extend stack to hold the values also
- The current top of stack is indicated by `ptr` top
- Suppose semantic rule $A.a = f(X.x, Y.y, Z.z)$ is associated with production $A \rightarrow XYZ$
- Before reducing XYZ to A, value of Z is in $val(top)$, value of Y is in $val(top - 1)$ and value of X is in $val(top - 2)$
- If symbol has no attribute then the entry is undefined

Bottom-up evaluation of S-attributed definitions

- Can be evaluated while parsing
- Whenever reduction is made, value of new synthesized attribute is computed from the attributes on the stack
- Extend stack to hold the values also
- The current top of stack is indicated by `ptr` top
- Suppose semantic rule $A.a = f(X.x, Y.y, Z.z)$ is associated with production $A \rightarrow XYZ$
- Before reducing XYZ to A, value of Z is in $val(top)$, value of Y is in $val(top - 1)$ and value of X is in $val(top - 2)$
- If symbol has no attribute then the entry is undefined
- After the reduction, top is decremented by 2 and state covering A is put in val(top)

Example: Calculator

$L \to En$	$print(val(top))$
$E \to E + T$	$val(ntop) = val(top-2) + val(top)$
$E \to T$	
$T \to T * F$	$val(ntop) = val(top-2) * val(top)$
$T \to F$	
$F \to (E)$	$val(ntop) = val(top-1)$
$F \to digit$	

Calculator

Input	State	Val	Production
3*5+4n			
*5+4n	digit	3	
*5+4n	F	3	F→digit
*5+4n	T	3	T→ F
5+4n	T*	3*	
+4n	T*digit	3*5	
+4n	T*F	3*5	F → digit
+4n	T	15	T→ T*F
+4n	E	15	E→ T
4n	E+	15-	
n	E+digit	15-4	
n	E+F	15-4	F→ digit
n	E+T	15-4	T→ F
n	E	19	E→ E+T

L-attributed definitions

- A syntax directed definition is L-attributed if each inherited attribute of $X_j (1 \leq j \leq n)$ at the right hand side of $A \rightarrow X_1 X_2 \cdots X_n$ depends only on

L-attributed definitions

- A syntax directed definition is L-attributed if each inherited attribute of $X_j (1 \leq j \leq n)$ at the right hand side of $A \rightarrow X_1 X_2 \cdots X_n$ depends only on
  - Attributes of symbols $X_1 X_2 \cdots X_{j-1}$ and

L-attributed definitions

- A syntax directed definition is L-attributed if each inherited attribute of $X_j (1 \leq j \leq n)$ at the right hand side of $A \rightarrow X_1 X_2 \cdots X_n$ depends only on
  - Attributes of symbols $X_1 X_2 \cdots X_{j-1}$ and
  - Inherited attribute of A

L-attributed definitions

- A syntax directed definition is L-attributed if each inherited attribute of $X_j (1 \leq j \leq n)$ at the right hand side of $A \rightarrow X_1 X_2 \cdots X_n$ depends only on
  - Attributes of symbols $X_1 X_2 \cdots X_{j-1}$ and
  - Inherited attribute of A
- $A \rightarrow LM$ L.i = f1(A.i)
  M.i = f2(L.s)
  A.s = f3(M.s)

L-attributed definitions

- A syntax directed definition is L-attributed if each inherited attribute of $X_j (1 \leq j \leq n)$ at the right hand side of $A \rightarrow X_1 X_2 \cdots X_n$ depends only on
  - Attributes of symbols $X_1 X_2 \cdots X_{j-1}$ and
  - Inherited attribute of A
- $A \rightarrow LM$ L.i = f1(A.i)
  M.i = f2(L.s)
  A.s = f3(M.s)
- $A \rightarrow QR$ R.i = f4(A.i)
  Qi = f5(R.s)
  A.s = f6(Q.s)

L-attributed definitions

- A syntax directed definition is L-attributed if each inherited attribute of $X_j (1 \leq j \leq n)$ at the right hand side of $A \rightarrow X_1 X_2 \cdots X_n$ depends only on
  - Attributes of symbols $X_1 X_2 \cdots X_{j-1}$ and
  - Inherited attribute of A
- $A \rightarrow LM$ L.i = f1(A.i)
  M.i = f2(L.s)
  A.s = f3(M.s)
- $A \rightarrow QR$ R.i = f4(A.i)
  Q.i = f5(R.s)
  A.s = f6(Q.s)

Translation Scheme

- A CFG where semantic actions occur within the RHS of production

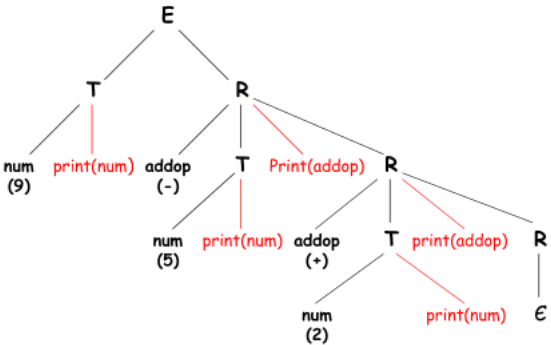
Translation Scheme

- A CFG where semantic actions occur within the RHS of production
- A translation scheme to map infix to postfix

Translation Scheme

- A CFG where semantic actions occur within the RHS of production
- A translation scheme to map infix to postfix
  $E \rightarrow TR$
  $R \rightarrow addop\ T \quad \{print(addop)\} \quad R|\epsilon$
  $T \rightarrow num \quad print(num)$

Parse tree for 9-5+2



- Assume actions are terminal symbols

- Assume actions are terminal symbols
- Perform depth first order traversal to obtain 9 5 – 2 +

- Assume actions are terminal symbols
- Perform depth first order traversal to obtain 9 5 – 2 +
- When designing translation scheme, ensure attribute value is available when referred to

- Assume actions are terminal symbols
- Perform depth first order traversal to obtain 9 5 – 2 +
- When designing translation scheme, ensure attribute value is available when referred to
- In case of synthesized attribute it is trivial.

- Assume actions are terminal symbols
- Perform depth first order traversal to obtain 9 5 – 2 +
- When designing translation scheme, ensure attribute value is available when referred to
- In case of synthesized attribute it is trivial.
- In case of both inherited and synthesized attributes?

- Assume actions are terminal symbols
- Perform depth first order traversal to obtain 9 5 – 2 +
- When designing translation scheme, ensure attribute value is available when referred to
- In case of synthesized attribute it is trivial.
- In case of both inherited and synthesized attributes?
  - $S \rightarrow __ A __ B __$

- Assume actions are terminal symbols
- Perform depth first order traversal to obtain 9 5 – 2 +
- When designing translation scheme, ensure attribute value is available when referred to
- In case of synthesized attribute it is trivial.
- In case of both inherited and synthesized attributes?
  - $S \rightarrow _ \ A \ _ \ B \ _$
  - An inherited attribute for a symbol on rhs of a production must be computed in an action before that symbol
  - $S \rightarrow A_i A B_i B S_s$

Example: Translation scheme for EQN

$S \rightarrow B$
$B \rightarrow B_1 B_2$
$B \rightarrow B_1$ sub B_2
$B \rightarrow text$
Attributes for the symbols

- pts
- ht

Example: Translation scheme for EQN

$S \rightarrow B$ $\quad\quad\quad\quad$ $B.pts = 10$

Example: Translation scheme for EQN

$S \rightarrow B \qquad B.pts = 10$

$B \rightarrow B_1 B_2 \qquad B_1.pts = B.pts$

Example: Translation scheme for EQN

$S \to B$	$B.pts = 10$
$B \to B_1 B_2$	$B_1.pts = B.pts$
	$B_2.pts = B.pts$

Example: Translation scheme for EQN

$S \rightarrow B$	$B.pts = 10$
$B \rightarrow B_1 B_2$	$B_1.pts = B.pts$
	$B_2.pts = B.pts$
$B \rightarrow B_1$ sub B_2	$B_1.pts = B.pts$

Example: Translation scheme for EQN

$S \rightarrow B$	$B.pts = 10$
$B \rightarrow B_1 B_2$	$B_1.pts = B.pts$
	$B_2.pts = B.pts$
$B \rightarrow B_1$ sub B_2	$B_1.pts = B.pts$
	$B_2.pts = shrink(B.pts)$
$B \rightarrow text$	

Example: Translation scheme for EQN

$S \rightarrow B$	$B.pts = 10$
	$S.ht = B.ht$
$B \rightarrow B_1 B_2$	$B_1.pts = B.pts$
	$B_2.pts = B.pts$
	$B.ht = max(B_1.ht, B_2.ht)$
$B \rightarrow B_1$ sub B_2	$B_1.pts = B.pts$
	$B_2.pts = shrink(B.pts)$
	$B.ht = disp(B_1.ht, B_2.ht)$
$B \rightarrow text$	$B.ht = text.h * B.pts$

Example: SDT Form

$S \rightarrow \{B.pts = 10\}$ B
$\{S.ht = B.ht\}$

Example: SDT Form

$S \rightarrow \{B.pts = 10\}$ B
$\quad \{S.ht = B.ht\}$
$B \rightarrow \{B_1.pts = B.pts\} \quad B_1$
$\quad \{B_2.pts = B.pts\} \quad B_2$
$\quad \{B.ht = max(B_1.ht, B_2.ht)\}$

Example: SDT Form

$S \rightarrow \{B.pts = 10\}$ B
$\quad \{S.ht = B.ht\}$
$B \rightarrow \{B_1.pts = B.pts\} \quad B_1$
$\quad \{B_2.pts = B.pts\} \quad B_2$
$\quad \{B.ht = max(B_1.ht, B_2.ht)\}$
$B \rightarrow \{B_1.pts = B.pts\} \quad B_1$ sub
$\quad \{B_2.pts = shrink(B.pts)\} \quad B_2$
$\quad \{B.ht = disp(B_1.ht, B_2.ht)\}$

Example: SDT Form

$S \rightarrow \{B.pts = 10\}$ B
$\{S.ht = B.ht\}$
$B \rightarrow \{B_1.pts = B.pts\}$ B_1
$\{B_2.pts = B.pts\}$ B_2
$\{B.ht = max(B_1.ht, B_2.ht)\}$
$B \rightarrow \{B_1.pts = B.pts\}$ B_1 sub
$\{B_2.pts = shrink(B.pts)\}$ B_2
$\{B.ht = disp(B_1.ht, B_2.ht)\}$
$B \rightarrow text$ $\{B.ht = text.h * B.pts\}$